

Correction-Linked Lifecycle Memory for Evolving Long-Horizon Dialogues

Anonymous Author(s)

Abstract

Long-running programming agents must remember decisions while avoiding constraints that were later corrected, renamed, reverted, or deleted. Static retrieval can surface obsolete statements, whereas deletion alone may discard the very correction needed to recover the current state. We introduce Correction-Linked Lifecycle Memory (CLLM), a bounded sidecar for an existing memory retriever. Write-time correction events create predecessor-successor links; query-time traversal redirects stale hits to their latest correction nodes; and a small policy grid is selected from controlled feedback without retraining the retriever or extractor. The controller is switchable and hard-falls back to the original result list on missing state, corruption, timeout, or capacity failure. We evaluate the method in MemoryCore using the public Memora benchmark as a methodological proxy for long programming sessions. The optimizer observes weekly/monthly feedback and is frozen before quarterly evaluation. Across 192 held-out forgetting-bearing questions, direct evidence FAMA improves by 2.68 points (95% persona-cluster CI: 1.47–4.13). On 50 frozen quarterly questions where the baseline actually retrieves stale evidence, answer-level FAMA improves from 0.334 to 0.401, a 6.78-point gain (95% CI: 2.12–12.32), with a 1.55% increase in injected tokens. The primary FAMA gate passes, but a predeclared secondary FAA gain of 10 points is missed (observed: 7.52), so the composite promotion gate remains failed. These results support correction-linked retrieval as a promising conditional mechanism, not a claim of validation on real programming-agent traffic.

1 Introduction

Interactive programming tasks evolve. A user may replace REST with GraphQL, rename an interface, invalidate a test expectation, revert an implementation plan, or switch environments. A memory system that retrieves both the old and new statements can confidently reintroduce a rejected constraint. Conversely, permanently deleting the old record loses provenance and may prevent the system from finding the correction that explains the current state.

Long-term agent memory research has studied hierarchical context management [3], dynamic organization [9], scalable extraction and consolidation [1], and temporal knowledge graphs [4]. Benchmarks such as LoCoMo [2] and LongMemEval [8] reveal persistent failures in temporal reasoning and knowledge updates. Memora makes forgetting explicit through invalidated-memory annotations and Forgetting-Aware Memory Accuracy (FAMA) [7]. Yet three practical questions remain: (1) whether obsolete retrieval can be detected before relying on end-task success, (2) whether retaining a correction link is better than filtering an old hit, and (3) whether a bounded policy can adapt from feedback while preserving a hard baseline fallback.

We address these questions in MemoryCore, an open-source layered memory service with L0 conversation retrieval and higher-level memories [6]. Our contributions are:

- a replaceable write-event interface and bounded lifecycle ledger that preserve the base FTS/vector engine;
- correction-linked retrieval that redirects stale predecessors to later update or deletion events instead of merely suppressing them;
- a feedback optimizer with a frozen weekly/monthly-quarterly split, capacity limits, decision logs, an off switch, and exact hard fallback;

- a protocol-versioned evaluation separating direct evidence, gold upper bounds, held-out adaptive results, and answer-level effects.

2 Related Work

MemGPT treats context as a hierarchy managed like virtual memory [3]. Mem0 dynamically extracts, consolidates, and retrieves user information, and A-MEM organizes notes into an evolving linked network [1, 9]. Zep/Graphiti represents changing facts in a temporally aware graph [4]. Our work is narrower: it does not replace the store or build a general knowledge graph. It adds bounded invalidation and successor edges directly above an existing candidate list.

LoCoMo evaluates very long multi-session dialogue [2]; LongMemEval isolates extraction, multi-session reasoning, temporal reasoning, updates, and abstention [8]. Memora is especially suitable here because its public protocol distinguishes memory-presence from forgetting-absence criteria [7]. We use it as a method-validation proxy, not as evidence about private programming interactions.

Feedback-driven agents such as Reflexion update behavior from verbal or scalar feedback without weight training [5]. Our optimizer follows the same broad test-time adaptation motivation, but evolves only a bounded retrieval policy (confidence and hop limit) under a fixed evaluation split.

3 Method

Let the base retriever return a ranked candidate pool $B(q) = (m_1, \dots, m_K)$. A write-time adapter emits correction events

$$e = (t_e, c_e, V_e^-, S_e, s_e),$$

where t_e is a monotonic sequence, $c_e \in [0, 1]$ is confidence, V_e^- contains invalidated values, S_e contains successor memory identifiers, and s_e records signal provenance. A predecessor memory m receives an edge to S_e when it predates e and contains a value in V_e^- .

At query time, CLLM examines candidates in base rank order. If no eligible outgoing edge meets confidence threshold τ , the candidate is unchanged. Otherwise, only the latest eligible correction event is followed. Traversal continues until a leaf, tombstone, hop budget H , expansion budget E , result budget k , or time budget T is reached. Duplicate leaves are removed and later base candidates refill the list. Cycles, missing successors, timeouts, malformed state, and capacity violations cause the entire call to return $B(q)_{1:k}$.

The policy is $\theta = (\tau, H, E, k, T)$. We enumerate a fixed grid over $\tau \in \{0.85, 0.95\}$ and $H \in \{1, 2, 4, 8\}$, with $E = 64$, $k = 5$, and $T = 10$ ms. On the optimization split, utility is

$$U(\theta) = Q(\theta) - \alpha C(\theta) - \beta F(\theta),$$

where Q is mean direct evidence FAMA, C is mean injected kilotokens, F is fallback rate, $\alpha = 0.01$, and $\beta = 1$. Ties prefer lower cost, fewer hops, and higher confidence. The selected policy is frozen before held-out evaluation.

4 Experimental Protocol

System and data. We use MemoryCore L0 SQLite FTS5 retrieval with a candidate pool of 30 and an injected limit of five. Memora revision `a6493188efc836d6511ed5e4163fe3ba87da30ff` contains 600 questions, 27,614 sessions, 10 personas, and weekly, monthly, and quarterly horizons. The aggregate data-manifest SHA-256 is `dfc82711...331bfc`. Only turns marked `share_memory` are indexed.

Leakage control. The ledger uses released write-time fields (session type, operation, operation details, and shared turns), never evaluation evidence. Weekly and monthly forgetting-bearing questions (188) optimize policy parameters. Quarterly questions are held out. Evaluation evidence scores outputs and defines a diagnostic *stale-exposed* population, but is invisible to the controller.

Evaluation	n	Base	CLLM	Delta [95% CI]
Direct FAMA, held-out forgetting	192	0.450	0.477	+0.027 [0.015, 0.041]
Direct FAMA, stale-exposed	61	0.091	0.184	+0.093 [0.059, 0.128]
Answer MPA, stale-exposed	50	0.388	0.440	+0.052 [0.004, 0.110]
Answer FAA, stale-exposed	50	0.866	0.941	+0.075 [0.050, 0.102]
Answer FAMA, stale-exposed	50	0.334	0.401	+0.068 [0.021, 0.123]

Table 1: Held-out results. Intervals use paired persona-cluster bootstrap.

Metrics. Memory Presence Accuracy (MPA) measures required current facts; Forgetting Absence Accuracy (FAA) measures avoidance of invalid facts. Following Memora,

$$\text{FAMA} = \max(0, \text{MPA} - \lambda(1 - \text{FAA})), \quad \lambda = \frac{N_f}{N_p + N_f}.$$

Direct evidence metrics use value and temporal-boundary alignment before answer generation. Answer-level metrics use a fixed GPT-4o-mini reader and a fixed Gemini-2.5-Flash-Lite judge with temperature zero. This cheaper batched-judge protocol is not directly comparable to Memora Table 3. We report paired 5,000-sample persona-cluster bootstrap intervals.

Frozen answer-level subset. Among 192 held-out forgetting-bearing quarterly questions, 61 have at least one stale Base Top-5 hit. Before answer generation, we hash-stratify five cases per persona, yielding 50 cases (43 recommending and seven remembering). The selection SHA-256 is `1e3ab7b2...dadaf1`.

5 Results

Table 1 reports the main comparisons. On all held-out forgetting-bearing questions, the direct proxy improves by 2.68 points. Conditional on actual stale exposure, the proxy gain is 9.29 points. At answer level, FAMA increases by 6.78 points with a confidence interval excluding zero. Both MPA and FAA improve, showing that correction links can recover current information rather than trading recall solely for forgetting.

The optimizer selects $\tau = 0.85$ and $H = 1$. Although the gold ablation requires transitive traversal, ledger construction links a predecessor to every detected later correction and the resolver chooses the latest eligible edge, allowing one-hop query-time redirection. Mean injected context changes from 144.52 to 146.76 tokens (+1.55%). Mean lifecycle computation is 0.018 ms in the answer-level run, and no evaluated call falls back.

The primary preregistered checks—FAMA gain of at least five points and a positive lower confidence bound—pass. The composite gate nevertheless remains failed because the observed FAA gain (7.52 points) is below the predeclared 10-point secondary threshold. We retain this failure rather than revising the threshold post hoc.

Ablation. Gold invalidation-only filtering raises FAA but yields only a 2.64-point answer FAMA gain with an interval crossing zero. Gold transitive successor traversal yields a 9.97-point gain (95% CI: 1.75–19.27). This contrast motivates preserving correction links rather than treating forgetting as deletion.

Safety behavior. Across all 600 questions, disabling CLLM produces zero candidate-ID mismatches against Base. Forced damaged-state and timeout tests also produce zero mismatches (600/600 each). The implementation bounds each group to 10,000 units, 5,000 events, and 50,000 edges, in addition to query-time budgets.

6 Limitations and Transfer

First, Memora is personalized dialogue, not a programming-agent benchmark. Internal programming sessions may contain branch-local truths, implicit corrections, code moves, and tool-grounded state changes that do

not quote the invalidated value. Second, the answer-level experiment is conditional on stale exposure and task-imbalanced toward recommendation. Third, it uses one batched judge. A predeclared three-judge replication stopped at 22/50 because of an external HTTP 402 credit error; partial results are excluded. Fourth, structured Memora operation metadata is stronger than typical weak conversational feedback. An internal adapter must calibrate signals such as explicit user correction, rename/refactor events, reverted plans, changed tests, and tool-confirmed deletion. Finally, no production Gateway integration is claimed; the current contribution is a sidecar module, adapter contract, and reproducible protocol.

7 Conclusion

Static retrieval can convert historical truth into current error. In this study, deletion alone is insufficient: the useful operation is to retain a bounded link from a stale predecessor to the correction that supersedes it. A label-free, feedback-selected controller improves held-out direct and answer-level FAMA at nearly constant context cost while preserving exact fallback behavior. The result justifies an internal programming-session evaluation, but not yet a broad claim about long-horizon coding agents.

References

- [1] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [2] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.
- [3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [4] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [5] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [6] TencentCloud. Tencetdb agent memory: Memorycore. <https://github.com/TencentCloud/TencentDB-Agent-Memory/tree/v2.0.0-beta.1/MemoryCore>, 2026.
- [7] Md Nayem Uddin, Kumar Shubham, Eduardo Blanco, Chitta Baral, and Gengyu Wang. From recall to forgetting: Benchmarking long-term memory for personalized agents. *arXiv preprint arXiv:2604.20006*, 2026.
- [8] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [9] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.